

Retrieval-Augmented Generation (RAG) Explained

Retrieval-Augmented Generation (RAG) is a way to make an AI chatbot smarter by letting it “look up” information before answering. In simple terms, a RAG system takes the user’s question, **searches a database of documents** for relevant facts, then feeds those facts into the language model to generate the reply. This means the model isn’t just guessing from its training; it uses up-to-date, domain-specific information to be more accurate ¹. For example, if you ask “Who is the Chief Minister of my state?”, a RAG chatbot finds that answer in its stored data (rather than just hallucinating), and includes it in the response ¹ ².

Core Technology Components

A RAG-based voice assistant involves several pieces working together:

- **Telephony / Call Handling:** A service (like Twilio Programmable Voice) answers the phone call and plays prompts. This front end connects the caller’s audio to our backend system.
- **Speech-to-Text (ASR):** The caller’s speech is sent to a speech recognition engine (e.g. Google Speech-to-Text or Azure Cognitive Services). This converts the spoken question into text for the AI to process.
- **Knowledge Base / Database:** We maintain a database of all government scheme information (from official sites, PDFs, etc.). The retrieval component searches this corpus.
- **Retrieval Engine:** This is the “R” in RAG. It uses a semantic search model (often based on text embeddings) to find the most relevant passages for the question. In practice, we embed the question and all documents into vectors, then use a vector database (like FAISS or Elasticsearch) to retrieve top-matching text chunks ³.
- **Language Model (LLM):** The retrieved passages plus the original question are given to an LLM (such as GPT-4 or an open-source transformer). The LLM’s job is to **generate** the final answer, combining the retrieved facts with its own language ability. The model effectively weaves the fresh data into a coherent answer ³.
- **Text-to-Speech (TTS):** Finally, the answer text is sent to a TTS engine (e.g. AWS Polly or Azure TTS) to convert it into spoken audio. That audio is played back to the caller.

Each component plays a clear role: ASR and TTS handle the voice interface, the retrieval engine finds facts, and the LLM “glues” the facts into an answer. (For example, Vonage notes this pipeline as: *speech-to-text* → *retrieve relevant info* → *LLM generate answer* → *text-to-speech* ⁴.)

Example Call Flow

1. **User asks a question:** The citizen calls and says, for example, “What documents do I need for the rural pension scheme?”.
2. **Convert speech to text:** The system sends the audio to an ASR service. It returns a text transcript like “what documents do I need for the rural pension scheme.”

3. **Retrieve relevant documents:** The retrieval engine takes that text, embeds it into a vector, and searches the indexed scheme documents. It returns the top passages (e.g. a paragraph about pension scheme eligibility and documents).
4. **Generate the answer:** These passages and the user's question are fed into the LLM. The model writes a clear answer such as, "To apply for the rural pension scheme, you need your Aadhaar card, proof of income, and land ownership papers." This answer is based on the retrieved facts.
5. **Convert to speech:** The text answer is sent to TTS, which produces audio in the user's language.
6. **Play response:** The assistant plays the answer audio on the call.

This flow (speech → text → retrieve → generate → speech) matches standard RAG voice chatbot pipelines ⁴ ³. In practice, each step is fast: the retrieval search is optimized to be quick (e.g. using FAISS) and the LLM returns a short answer. The result is that the user hears a natural, fact-based response.

Accuracy, Latency, and Cost Trade-offs

Using RAG has clear benefits but also trade-offs:

- **Accuracy (Factuality):** RAG significantly improves factual accuracy. By pulling in real information from the schemes database, it grounds answers in reality and reduces hallucinations. In fact, studies find that retrieval-based and RAG systems score *very high* on factual accuracy, whereas pure generative chatbots without retrieval were only *moderate* due to more errors ⁵. Rule-based systems (fixed scripts) also have very high accuracy, but they only answer exactly what's scripted. RAG combines reliability with flexibility.
- **Latency (Speed):** Each added step (ASR, retrieval, LLM, TTS) adds delay. Voice calls are sensitive to lag: telecom standards aim for ~100 ms one-way latency ⁶. RAG's retrieval and generation make this hard. (Indeed, one analysis warns that achieving even 150 ms is nearly impossible with RAG due to so many components ⁶.) In practice, we keep models small and optimize search to answer in maybe 1–2 seconds. That is usually acceptable for a phone Q&A, but it's noticeably slower than a simple rule-based IVR which could reply instantly.
- **Cost:** Running an LLM (GPT-4 or similar) for each query incurs API costs (often per token). RAG requires extra infrastructure (vector DB hosting, query costs). This adds up. In contrast, a rule-based or retrieval-only system (with just a database query or search) has far lower operational cost. One blog notes that LLMs introduce significant *per-token costs and latency*, along with privacy and engineering overhead ⁷. So RAG is costlier, especially if call volume is high.
- **Offline Use:** If internet or cloud access is limited (rural areas), RAG can be challenging since it relies on cloud LLM APIs or heavy servers. In contrast, an offline rule-based bot (with all logic on-device) or a lightweight on-device model could work without connectivity. RAG systems generally assume a cloud backend, so for fully offline use one might prefer a smaller local model or a pure retrieval approach.

In summary, RAG trades extra complexity for answer quality. It gives **high accuracy and flexibility** but at the expense of higher cost and slower responses. Simpler architectures (rule-based or retrieval-only) are faster and cheaper, but can only handle the knowledge explicitly programmed into them.

Starter Resources & Experiments

To try RAG yourself, start small and local:

- **Tutorials:** The [GeeksforGeeks RAG guide](https://www.geeksforgeeks.org/nlp/rag-architecture/) outlines the whole process with code examples ¹ ³ . It even shows how to install FAISS and LangChain for vector search and LLM use ⁸ .
- **Vector Search Demo:** You can build a mini Q&A without any LLM. For example, Elasticsearch Labs shows a Streamlit app using their ELSEr semantic search. The user asks a question, and the app directly returns the most relevant sentence from indexed documents ⁹ . This is nearly RAG but *without* generation – it just highlights the answer. It's fast and avoids LLM costs, although the answer can be terse.
- **Libraries:** Python libraries like Hugging Face's Transformers and LangChain simplify RAG. Try creating a small vector index with FAISS and a tiny LLM (e.g. GPT-2) on your PC. Feed in a couple of scheme documents, then ask it questions. The transformer's `.generate()` call plus a FAISS - based retrieval is a solid RAG prototype ⁸ .
- **Tool/Function Calling:** Another modern approach is letting a model decide when to call a tool. For example, you could allow a lightweight LLM (like GPT-3.5) to use function-calling to either query a database or perform a search as needed ¹⁰ . This blends RAG with "agentic" decision-making and could be a fun experiment.
- **Rule-based Test:** As a control experiment, implement a simple script-based bot. For instance, code an "if the user says this keyword, reply with that sentence" system. Tools like Rasa or Microsoft Bot Framework let you define Q&A pairs. Measure how often this covers user questions vs how often it fails. This highlights the coverage gap RAG aims to fill.
- **Evaluation:** Collect a small set of typical user questions (maybe from friends/family) about schemes. Try answering them with your RAG prototype and with a simpler system (e.g. a keyword search or template replies). Compare answer correctness and response time. This hands-on approach will show the RAG benefits and its demands firsthand.

Sources: For RAG concepts and workflows we used educational resources ¹ ³ . Voice-chatbot pipeline examples come from engineering blogs ⁴ . Performance trade-offs were informed by analysis of RAG systems versus simpler methods ⁵ ⁷ . These provide the background for the above explanation and recommendations.

¹ ² ³ ⁸ RAG Architecture - GeeksforGeeks

<https://www.geeksforgeeks.org/nlp/rag-architecture/>

⁴ ⁶ Reducing RAG Pipeline Latency for Real-Time Voice Conversations

<https://developer.vonage.com/en/blog/reducing-rag-pipeline-latency-for-real-time-voice-conversations>

⁵ Frontiers | Evaluating chatbot architectures for public service delivery: balancing functionality, safety, ethics, and adaptability

<https://www.frontiersin.org/journals/political-science/articles/10.3389/fpos.2025.1601440/full>

⁷ ⁹ RAG without LLMs: Building Q&A with ELSEr & semantic search - Elasticsearch Labs

<https://www.elastic.co/search-labs/blog/rag-without-llms>

¹⁰ What are the best RAG Alternatives? - Fingoweb

<https://www.fingoweb.com/blog/what-are-the-best-rag-alternatives/>